

If you enter a '?' in the REPL, it will switch to the HELP mode. In this mode, you may enter keywords and search for information on them. In the HELP mode, the prompt changes to yellow.

```
help?> peakflops()
... peakflops(n; parallel=false)
```

'peakflops' computes the peak flop rate of the computer by using double precision :`func: Base.LinAlg.BLAS.gemm!`. By default, if no arguments are specified, it multiplies a matrix of size 'n x n', where 'n = 2000'. If the underlying BLAS is using multiple threads, higher flop rates are realised. The number of BLAS threads can be set with `blas_set_num_threads(n)`.

If the keyword argument 'parallel' is set to 'true', 'peakflops' is run in parallel on all the worker processors. The flop rate of the entire parallel computer is returned. When running in parallel, only one BLAS thread is used. The argument 'n' still refers to the size of the problem that is solved on each processor.

The code above provides helpful information regarding the `peakflops()`.

To enter into shell mode, press ';'. From the shell mode, you can execute shell commands, as shown in Figure 4. In the shell mode, the prompt changes to red.

The REPL provides many more functions. A few popular functions and their descriptions are provided in Table 1.

The power of Julia

An interesting introduction to Julia by one of its co-creators, Viral Shaw, is available at <https://www.youtube.com/watch?v=6foFiKMXFk>. The illustration provided in this video enables us to understand and appreciate the power of Julia.

As stated earlier, Julia code runs very fast. Let's prove that with a simple but powerful example:

```
function test(n::Array)
    sum = 0.0
    for i = 1:length(n)
        sum+=n[i]
    end
    println(sum)
end

num = randn(10^7)
@time test(num)
```

The code above has a simple function to navigate through an array and find the sum of elements. The real power of Julia lies in the scalability. The code has a statement to generate 10 to the power of seven (10 million) random numbers (with normal distribution).

```
num = randn(10^7)
```

REPL function / macros	Description
<code>versioninfo()</code>	Provides the information regarding the installed Julia version as well as platform-specific information: <pre>julia> versioninfo() Julia Version 0.4.6 Commit 2e358ce (2016-06-19 17:16 UTC) Platform Info: System: Linux (x86_64-linux-gnu) CPU: Intel(R) Core(TM) i5-2400 CPU @ 3.10GHz WORD_SIZE: 64 BLAS: libopenblas (NO_LAPACKE DYNAMIC_ ARCH NO_AFFINITY Sandybridge) LAPACK: liblapack.so.3 LIBM: libopenlibm LLVM: libLLVM-3.3</pre>
<code>edit(file_name)</code>	Edits the file with name <code>file_name</code>
<code>@edit fn_name()</code>	Edits the definition of built-in function <code>fn_name()</code> . Julia allows you to edit the definition of built-in functions as well.
<code>@time</code>	Provides timing information for the command executed. For example: <pre>julia> @time randn(10^4) 0.000131 seconds (7 allocations: 78.359 KB)</pre>
<code>@code_native</code>	Provides native code generated for the execution of the function. For example: <pre>julia> @code_native 45+34 .text Filename: int.jl Source line: 8 pushq %rbp movq %rsp, %rbp Source line: 8 addq %rsi, %rdi movq %rdi, %rax popq %rbp ret</pre>

Table 1: Handy REPL functions

The 10 to the power of seven elements array is supplied as an input to the function. Julia is capable of looping through the 10 million elements array and finding the sum in 0.016 seconds.